FYP Refinement

Scheduling parallel subtasks locally & across a MEC system, from the application level

- Suppose task T is made up of n parallelisable subtasks, i.e. $T = T_1, T_2, \ldots, T_n$
- Initially, we assume each T_i is heterogenous

- An edge cluster has p_E "processors" (e.g. cores) that can all execute simultaneously
- Cloud has p_C processors (initially assumed to be $\gg p_E$ and $\geq n$, so that latency is ~0)
- A user device itself has p_U processors (assumed to be $\gg p_E$)

- An edge cluster can be "under heavy load" if busy with other tasks
- Some processors (in the same cluster) may be under a heavier load than others
- In such conditions, the latency per subtask per processor will be higher
- A user device can also be slowed, e.g. due to CPU utilisation or thermal throttling

Initially, we assume roughly that a subtask has one latency for an idle processor, and another latency for a processor under heavy load. For example:

Latency per subtask per processor	Idle	Under heavy load
User device	30 ms	90 ms
Edge cluster (5 ms RTT)	15 ms (+5 = 20 ms)	70 ms (+5 = 75 ms)
Cloud (20 ms RTT)	3 ms (+20 = 23 ms)	*Assumed not possible*

- First, we consider the decision process at the individual subtask (T_i) level
- The user device must choose between "local" and "offload to the system"
- If handed to the edge cluster, then the edge must choose between itself and the cloud
- Choices (scheduling) are decided based on live, dynamic information
- E.g. after scheduling 100 subtasks locally, the CPU may now be under heavy load

- That leads to the harder scheduling problem: can the total overall latency (of T) be improved by distributing many subtasks (of one main task) across the 3 nodes?
- At each level (user, edge, cloud), we need to know how many of the p processors are "idle" / can execute a subtask at an acceptable latency
- For each of those processors, we then need to work out how many subtasks could be added simultaneously to that processor without affecting performance
- These two results can be used as parameters to a distributing-scheduling algorithm

---

I'd build a simulator, and experiments involving different size tasks (i.e. different n), many users, many edges, and more variations can be used to design, develop and test the algorithm

---

Possible extensions / ways to make the project more complex:

- What if information (i.e. idle vs. busy) of the MEC network is delayed / inaccurate?
- What if instead of two fixed states "idle" and "busy", we used continuous values
- What if subtasks T_i and T_j aren't homogenous? Are there dependencies between them?
- Can other QoE measures other than latency be considered (e.g. battery consumption)

One issue to be addressed is the architecture: where is/are the decision/s being made?

Solution 1: Naïve unrealistic approach

- There is a centralised omniscient scheduler with access to live, fully accurate telemetry and RTT data from the cloud, edge, and user device
- The algorithm should then (if it's any good) be close to the optimal solution

Solution 2: Two-step approach

- The local device has some limited telemetry and RTT data about the edge, and possibly some even more limited data about the cloud, both of which may be outdated
- It has perfect accurate data about itself
- It estimates how many of the n subtasks "the MEC system" as a whole should get
- The local device ultimately splits the n subtasks in two: k get sent to the edge, and $(n-k)$ get processed locally. The device is responsible for executing the local sub-tasks in parallel across however many of the p_U processors are idle enough to take them

- The edge cluster has fully accurate data about itself, and fairly accurate cloud data
- There are k subtasks received at the edge. The edge performs a (probably very similar) algorithm to split these into two: Some sent to the cloud, and some done here
- If the local device's estimate was wrong, and performing more subtasks there would've been better, the overhead has already been lost in sending the k subtasks

Solution 3: All at the edge

- The entire scheduling split is decided at the edge server
- Apart from trivially small cases (in which case all computation is done locally), the local device sends a small message containing its local capabilities, and details of T
- The edge then has enough accurate information to split the n subtasks into three
- It sends back the plan to the local device
- The local device keeps k subtasks according to the plan, and sends the remaining $(n-k)$ to the edge to do, possibly alongside the cloud (indifferent to the local device)
- This should be a more optimal solution than *Solution 2*, but it introduces a new overhead of sending and receiving the small messages at the beginning (equal to an additional $\mathrm{RTT}_{\mathrm{Local}\rightarrow\mathrm{Edge}}$), but depending on edge proximity may be negligible